



INSTITUTO POLITECNICO NACIONAL



ESCUELA SUPERIOR DE COMPUTO

Sistemas Emebebidos

Sistema IoT para control de acceso y registro de ESTACIONAMIENTO PARA MOTOCICLETAS (vía lectura automática de placas)

Alumnos:

Bonilla Ramírez Josué Eleazar

Jimenez Meza Ana Harumi

Pedroza Villagomez Emir

Ramirez Claudia Emilia

Grupo: 6CM3

Profesor: Miguel Angel Aleman Arce

Repositorio:[**https://github.com/HARUSITA/IoT_Proyect**](https://github.com/HARUSITA/IoT_Proyect)

Fecha: 05/01/2026

Índice

1. 1. Resumen del proyecto
2. 2. Alcance y restricciones
3. 3. Requisitos
4. 4. Tecnologías y herramientas
5. 5. Arquitectura del sistema
6. 6. Componentes y funcionamiento
7. 7. Flujos de operación
8. 8. Mensajería MQTT (topics y payloads)
9. 9. Estructura del repositorio
10. 10. Instalación y ejecución (PC / Raspberry / ESP32)
11. 11. Pruebas y validación
12. 12. Solución de problemas (troubleshooting)
13. 13. Seguridad, privacidad y mantenimiento
14. 14. Mejoras futuras

1. Resumen del proyecto

IoT_Project implementa un sistema de registro y validación de acceso para un estacionamiento de motocicletas. El sistema automatiza la lectura de placas (ANPR) y consulta una base de datos para determinar si la motocicleta está autorizada. El resultado se refleja en un dashboard (panel web) y/o se devuelve a los dispositivos IoT para notificar el estado del acceso (permitido / denegado).

El enfoque es: identificar la motocicleta por placa y registrar/validar el acceso.

2. Alcance y restricciones

2.1 Alcance

- Lectura automática de placa de motocicleta (cámara + ANPR).
- Publicación del evento de lectura hacia un servidor central (Raspberry Pi) mediante MQTT.
- Validación contra base de datos MySQL (placas autorizadas).
- Registro del acceso (fecha/hora, placa, resultado, punto de entrada).
- Dashboard web para visualizar registros y administrar el padrón de motos (altas/bajas/consulta).

2.2 Fuera de alcance (omitido)

- Reconocimiento facial o identificación por credenciales.
- Cobro de estacionamiento o pagos.

3. Requisitos

3.1 Requisitos funcionales

- RF-01: El sistema debe capturar una imagen (o stream) de la placa de la motocicleta en el punto de acceso.
- RF-02: El sistema debe extraer la placa (texto) mediante ANPR/OCR.
- RF-03: El texto de la placa debe publicarse al servidor central mediante MQTT.
- RF-04: El servidor central debe validar la placa contra la base de datos.
- RF-05: El sistema debe registrar cada intento de acceso (permitido/denegado) con marca de tiempo.
- RF-06: El dashboard debe permitir consultar bitácoras y administrar el padrón de motocicletas autorizadas.

3.2 Requisitos no funcionales

- RNF-01: Respuesta de validación en pocos segundos (meta: < 3 s en red local).
- RNF-02: Trazabilidad: cada lectura debe quedar registrada, incluso si fue denegada.

- RNF-03: Disponibilidad: el broker MQTT y el servicio de validación deben iniciar automáticamente en Raspberry Pi.
- RNF-04: Seguridad básica: credenciales de BD y WiFi no deben quedar hardcodeadas en repositorios públicos.
- RNF-05: Portabilidad: dashboard ejecutable en Windows/Linux con Node.js.

4. Tecnologías y herramientas

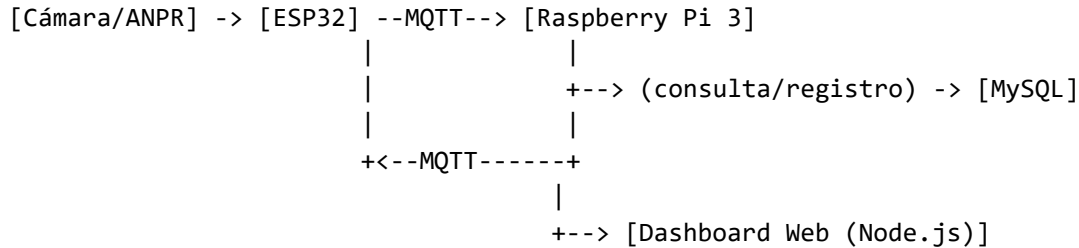
Componente	Tecnología	Uso
Dashboard / Backend	Node.js	Servidor web, API y lógica de registro/consulta
Base de datos	MySQL (puerto 3306)	Padrón de motos y bitácora de accesos
Mensajería	MQTT (en Raspberry Pi)	Comunicación en tiempo real entre ESP32 y servidor
Edge / Gateway	Raspberry Pi 3	Broker MQTT + servicio de validación + orquestación
Nodo de acceso	ESP32	Lectura/captura y envío de placa, recepción de respuesta
Lectura de placa	Cámara + ANPR/OCR	Conversión de imagen a texto de placa

5. Arquitectura del sistema

A nivel lógico, el sistema se divide en 3 capas:

- Capa IoT (Punto de acceso): ESP32 + cámara/lector de placas.
- Capa Edge/Gateway: Raspberry Pi 3 (broker MQTT + servicio validador).
- Capa Aplicación: Dashboard web (Node.js) + Base de datos (MySQL).

Diagrama lógico (texto):



6. Componentes y funcionamiento

6.1 Lector de placas (cámara + ANPR)

El lector de placas es el subsistema encargado de obtener el texto de la matrícula. La cámara toma una imagen enfocada a la placa; posteriormente, un algoritmo ANPR/OCR detecta la región de la placa y reconoce los caracteres. El resultado final debe ser un string normalizado (ej. sin espacios, en mayúsculas).

- Recomendaciones prácticas:
 - Iluminación controlada (evita reflejos).
 - Distancia y ángulo constantes (marcar zona de detención).
 - Normalización: quitar guiones/espacios y forzar mayúsculas.
 - Validación de formato (longitud mínima/máxima) antes de publicar.

6.2 ESP32 (nodo de acceso)

El ESP32 actúa como controlador del punto de entrada. Sus responsabilidades típicas son:

- Conectarse a la red WiFi.
- Conectarse al broker MQTT (normalmente alojado en la Raspberry Pi).
- Enviar el texto de placa detectada al topic correspondiente.
- Escuchar la respuesta del servidor (permitido/denegado) para mostrar un estado (LED, buzzer o pantalla).
- Entradas/salidas sugeridas:
 - Entrada: señal de “placa detectada” (por cámara/ANPR o por módulo externo).
 - Salida: LED verde/rojo (o display) para indicar permitido/denegado.
 - Salida opcional: buzzer corto para confirmación.

6.3 Raspberry Pi 3 (broker + servicio validador)

La Raspberry Pi es el centro de orquestación. Normalmente ejecuta:

- Un broker MQTT (por ejemplo Mosquitto).
- Un servicio (script) que se suscribe a los topics de lectura de placas.

- Lógica de validación: consulta a MySQL y decide el resultado.
- Publicación del resultado al ESP32 (topic de respuesta).
- Registro de bitácora (en BD) para trazabilidad.
- Ventajas de usar Raspberry como gateway:
 - Reduce carga en el ESP32 (no necesita conexión directa a BD).
 - Permite consolidar múltiples accesos (entrada 1, entrada 2, etc.).
 - Facilita monitoreo y reinicio automático (systemd).

6.4 Dashboard Web (Node.js) + MySQL

El dashboard permite administrar y visualizar la operación: padrón de motocicletas autorizadas y bitácora de accesos. El backend (Node.js) se conecta a MySQL (puerto 3306) usando un driver (por ejemplo mysql2) para ejecutar consultas y alimentar la interfaz.

En tu ejecución local aparece el mensaje “Dashboard Modal listo”, lo cual indica que el servidor arranca y luego intenta conectar a MySQL. El error reportado:
Access denied for user 'root'@'localhost' (using password: NO)
significa que el proyecto intenta conectarse con el usuario root SIN contraseña.

7. Flujos de operación

7.1 Validación de acceso (evento principal)

15. 1) La motocicleta se coloca en la zona de lectura.
16. 2) La cámara captura la imagen y el ANPR extrae la placa (texto).
17. 3) El ESP32 publica la placa al broker MQTT (Raspberry Pi).
18. 4) El servicio en Raspberry recibe el mensaje, normaliza la placa y consulta MySQL.
19. 5) El servicio decide el resultado: AUTORIZADO o NO AUTORIZADO.
20. 6) El servicio registra el evento en la bitácora.
21. 7) El servicio publica la respuesta al topic de respuesta del ESP32.
22. 8) El ESP32 muestra el estado (LED/display).

7.2 Administración del padrón (dashboard)

- Alta de motocicleta: guardar placa (normalizada) y datos básicos del propietario (si aplica).
- Baja/suspensión: marcar como inactiva para que los accesos se denieguen.
- Consulta: buscar por placa y revisar historial de accesos.

8. Mensajería MQTT (topics y payloads)

La comunicación MQTT se recomienda estructurarla por ubicación y tipo de evento. Ejemplo (ajústalo a tu implementación):

Tipo	Topic (ejemplo)	Payload (ejemplo)
Lectura de placa	escom/parking/motos/entrada1/placa	{ "placa": "ABC123", "ts": "2026-01-05T20:00:00-06:00" }
Respuesta	escom/parking/motos/entrada1/respuesta	{ "placa": "ABC123", "estado": "AUTORIZADO", "motivo": "OK" }
Heartbeat	escom/parking/motos/entrada1/heartbeat	{ "device": "esp32-entrada1", "uptime_s": 12345 }

- Buenas prácticas MQTT:
 - Usar QoS 1 para eventos de acceso (al menos una entrega).
 - Retained OFF para lecturas de placa (son eventos, no estado persistente).
 - Incluir timestamp y un identificador de punto de entrada.
 - Validar/filtrar mensajes duplicados en el servidor si QoS 1 reintenta.

9. Estructura del repositorio

En el repositorio se identifican al menos estas carpetas en la raíz:

- IoT_Proyect/ (aplicación principal: dashboard/backend)
- codigoESP32_arduinoIDE/ (firmware para ESP32 en Arduino IDE)
- codigoRaspBerryPi3/ (scripts/servicios para Raspberry Pi 3)
- package-lock.json (dependencias Node.js)

10. Instalación y ejecución (PC / Raspberry / ESP32)

10.1 Backend/Dashboard (PC con Windows)

- Prerrequisitos:
 - Node.js instalado (se recomienda LTS).
 - MySQL instalado y corriendo en localhost:3306.
 - Git para clonar/actualizar el repositorio.
- Pasos sugeridos:

23. 1) Abrir el proyecto en VS Code (File > Open Folder) apuntando a la carpeta del repo.
24. 2) Instalar dependencias: `npm install`
25. 3) Configurar la conexión a MySQL (usuario, contraseña, nombre de BD).
26. 4) Crear/importar el esquema de la BD (si el repo incluye un `.sql`, importarlo en MySQL).
27. 5) Ejecutar el servidor: `node app.js` (o el script equivalente en `package.json`).
28. 6) Abrir el navegador con la URL que imprima la consola (o típicamente `http://localhost:PORT`).

10.2 Cómo corregir el error de MySQL “using password: NO”

Tu error indica que Node intenta conectarse con root SIN contraseña. Tienes 3 opciones (elige una):

- Opción A (recomendada): crear un usuario propio para el proyecto.
 - 1) En MySQL, crea usuario y permisos para la BD del proyecto.
 - 2) Actualiza el archivo de configuración del proyecto (donde se define `host/user/password/db`).
- Opción B: configurar contraseña para root y colocarla en el proyecto.
 - Asegúrate de no subir credenciales a GitHub (usa variables de entorno o un archivo local ignorado por git).
- Opción C (NO recomendada): habilitar root sin contraseña.
 - Solo para pruebas locales; es inseguro y puede romper políticas de MySQL.

10.3 Raspberry Pi 3 (broker MQTT + validador)

- Configuración típica:
 - Instalar Mosquitto (broker MQTT) y habilitar el servicio.
 - Configurar un usuario/contraseña MQTT si la red no es totalmente confiable.
 - Configurar IP fija o reservación DHCP (para que el ESP32 siempre encuentre el broker).
 - Instalar dependencias del script validador (Python o Node, según el repo).
 - Crear un servicio systemd para que el validador arranque al encender.

10.4 ESP32 (Arduino IDE)

- Pasos típicos para compilar y cargar el firmware:
 - Instalar Arduino IDE.
 - Agregar soporte de placas ESP32 (Boards Manager).
 - Abrir el sketch en `codigoESP32_arduinoIDE/`.
 - Configurar SSID/Password WiFi y la IP/host del broker MQTT (Raspberry).
 - Seleccionar el puerto COM del ESP32 y cargar (Upload).

- Verificar en el Serial Monitor que se conecte a WiFi y a MQTT.

11. Pruebas y validación

- Checklist recomendado para validar el sistema end-to-end:
 - MQTT: el ESP32 publica un mensaje de prueba y la Raspberry lo recibe.
 - BD: existe una tabla/padrón con al menos 2 placas: una autorizada y una no autorizada.
 - Validador: al recibir una placa, responde correctamente AUTORIZADO/NO AUTORIZADO.
 - Bitácora: cada evento se inserta con timestamp y resultado.
 - Dashboard: muestra registros y permite altas/bajas.

12. Solución de problemas (troubleshooting)

Síntoma	Qué revisar / solución
Error MySQL 1045 (Access denied)	Revisar usuario/contraseña en la config; verificar que MySQL esté corriendo y que el usuario tenga permisos en la BD.
El servidor Node arranca pero no abre el dashboard	Verificar el puerto (PORT). Si está ocupado, cambia el puerto. Revisa la consola por la URL de escucha.
ESP32 no conecta a MQTT	Confirmar IP del broker, SSID/Password correctos, firewall en Raspberry, y que Mosquitto esté activo.
Placa ilegible o con falsos positivos	Mejorar iluminación, ángulo, resolución; aplicar normalización y validación de formato antes de enviar.

13. Seguridad, privacidad y mantenimiento

- Tratar la placa como dato personal: restringir acceso a la bitácora.
- No guardar credenciales en texto plano dentro de commits: usar .env, variables de entorno o archivos ignorados por git.
- Hacer respaldos periódicos de MySQL (padrón y bitácora).
- Registrar logs de MQTT/validador para diagnóstico (rotación de logs).

14. Mejoras futuras

- Soporte para múltiples entradas/salidas (entrada1, entrada2, etc.) con configuración por archivo.
- Lista blanca local en Raspberry (cache) para validar aun si MySQL está temporalmente caído.
- Dashboard con métricas: accesos por hora, placas más frecuentes, intentos denegados.
- Seguridad MQTT (TLS) y autenticación fuerte en red abierta.